

Task 13 — Twenty CRM on AWS EC2 with S3 Storage Backend

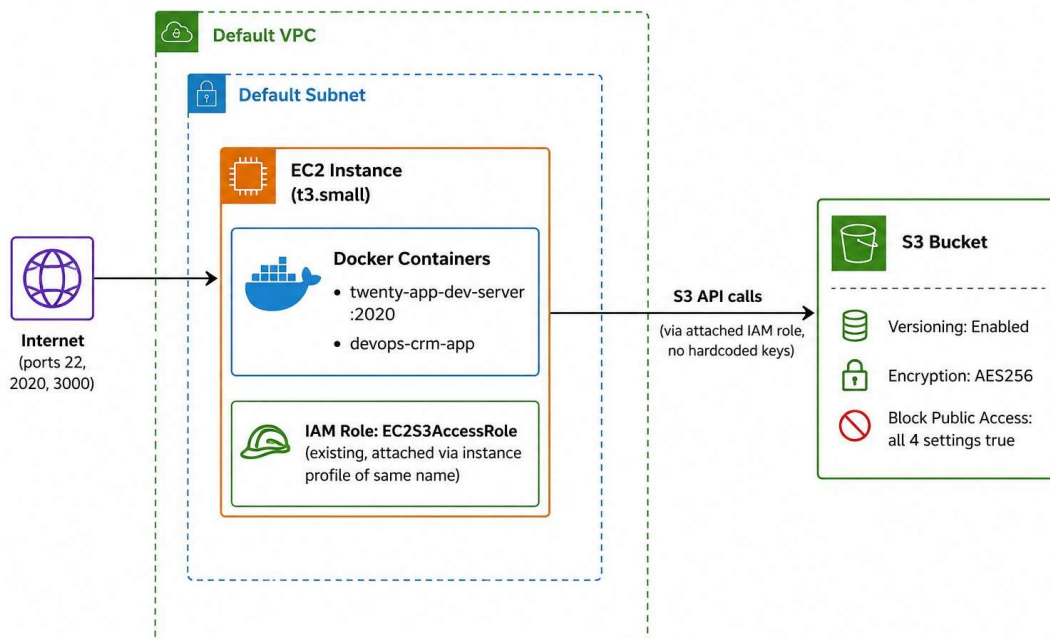
1. Executive Summary

This document describes the Terraform-based deployment of Twenty CRM on an AWS EC2 instance using Amazon S3 as the application storage backend. The solution uses the AWS account's existing default VPC, an approved Amazon Machine Image (AMI), a t3.small EC2 instance, and the pre-existing EC2S3AccessRole for S3 access. No new IAM users, roles, or policies are created.

The deployment is designed to be reproducible, secure by default, and verifiable through Terraform outputs and AWS CLI commands. The EC2 instance bootstraps Docker and the application automatically through cloud-init user data, while the application obtains S3 credentials through the attached IAM instance profile rather than hardcoded AWS access keys.

2. Architecture

The solution consists of a single EC2 instance running the Twenty CRM Docker application inside the default VPC. The instance exposes the required application ports and communicates with Amazon S3 using the permissions provided by the pre-existing IAM role.



3. Terraform Project Structure

File	Purpose
versions.tf	Pins the Terraform CLI and AWS provider versions and configures the AWS provider region.
variables.tf	Defines configurable inputs and validates the task's hard constraints.
data.tf	Looks up the existing default VPC, subnets, and caller identity.
s3.tf	Creates and configures the S3 bucket, versioning, encryption, and public-access blocking.
ec2.tf	Creates the security group and EC2 instance and references the existing IAM instance profile.
user_data.sh.tpl	Bootstraps Docker, clones the application repository, creates environment variables, and starts the application.
outputs.tf	Exposes deployment details such as instance ID, IP, DNS, bucket name, and network IDs.
terraform.tfvars	Contains deployment-specific variable values; should normally remain outside version control.

4. Detailed Terraform Components

4.1 versions.tf

The Terraform CLI is constrained to version 1.5.0 or later, while the AWS provider is constrained to the 5.x release family. The AWS provider uses the `aws_region` variable. Version constraints improve reproducibility and reduce the risk of unexpected behavior from future provider releases.

4.2 variables.tf

- `aws_region` — restricted to `us-east-1`.
- `instance_type` — restricted to `t3.small`.
- `ami_id` — restricted to the two approved AMI IDs.
- `existing_iam_role_name` — defaults to `EC2S3AccessRole`.
- `key_pair_name` — optional EC2 key pair used for SSH access.
- `ssh_ingress_cidr` — controls the source CIDR permitted to connect to TCP/22.
- `s3_bucket_name` — deployment-specific globally unique S3 bucket name.
- `project_name` and `owner` — used for resource naming and tagging.
- `github_repo_url` and `github_branch` — identify the application repository and branch used during bootstrap.

4.3 data.tf

Existing AWS resources are discovered through data sources instead of being created. The default VPC and its subnets are used for networking, and the current AWS caller identity is available for reference or tagging. The IAM role is intentionally referenced by its known instance-profile name rather than looked up through `aws_iam_role`, avoiding an unnecessary `iam:GetRole` permission requirement.

4.4 s3.tf

- Creates the S3 bucket.
- Enables bucket versioning.
- Enforces AES-256 server-side encryption by default.

- Enables all four S3 Block Public Access settings.
- Applies project and owner tags.

4.5 ec2.tf

- Creates a security group allowing SSH (22), Twenty CRM (2020), and frontend traffic (3000).
- Allows outbound traffic required by the application and bootstrap process.
- Launches one t3.small EC2 instance using an approved AML.
- Places the instance in the selected subnet within the default VPC.
- References the existing EC2S3AccessRole instance profile by name.
- Renders user_data.sh.tpl with repository, branch, bucket, and region values.
- Uses an explicit dependency on the S3 configuration so the bucket is available before the instance bootstraps.

4.6 user_data.sh.tpl

The bootstrap script runs during the first EC2 boot. It installs Docker Engine and Docker Compose, clones the requested GitHub branch, writes the AWS region and S3 bucket name to a .env file, and runs docker compose build followed by docker compose up -d.

AWS access keys are not copied to the instance. The application is expected to obtain temporary AWS credentials through the EC2 instance metadata/IAM role mechanism and use those credentials for S3 API calls.

4.7 outputs.tf

- EC2 instance ID
- EC2 public IP
- EC2 public DNS
- Application URL on port 2020
- S3 bucket name and ARN
- VPC ID
- Subnet ID
- IAM instance profile used

4.8 terraform.tfvars

Deployment-specific values are populated from terraform.tfvars.example. Although this task does not store credentials in the file, real-world projects should keep environment-specific tfvars files out of source control where appropriate.

5. Docker Compose S3 Integration

The application repository's Docker Compose configuration was updated so that Twenty CRM uses the provisioned S3 bucket rather than local Docker volumes for the relevant storage backend.

```
environment:
  - STORAGE_TYPE=S_3
  - STORAGE_S3_REGION=${AWS_REGION}
  - STORAGE_S3_NAME=${S3_BUCKET_NAME}
```

No AWS access key or secret access key is configured in Docker Compose. Authentication is provided by the EC2 instance's attached IAM role.

6. Deployment Procedure

1. Create the Terraform project with one file per infrastructure concern and the templated bootstrap script.
2. Configure the AWS CLI locally with an authorized IAM identity and set the region to us-east-1.
3. Create terraform.tfvars from the example file and populate the bucket name, application branch, key pair name, and other required values.
4. Run terraform init to download the pinned AWS provider.
5. Run terraform validate to check Terraform syntax, variable validation, and internal references.
6. Run terraform plan and review the proposed resources before applying changes.
7. Run terraform apply to provision the S3 resources, security group, and EC2 instance.
8. If SSH verification is required, create or select the approved EC2 key pair, set key_pair_name, and re-apply the configuration.
9. Use Terraform outputs and AWS CLI commands to verify the infrastructure configuration.
10. Open the Twenty CRM endpoint at http://<public_ip>:2020 after the instance finishes bootstrapping.
11. Commit the Terraform project and Docker Compose changes, push the task branch, and open a pull request into devops-crm-project.
12. After testing, run terraform destroy and confirm that the test infrastructure has been removed.

7. Verification and Validation

The deployment was verified through Terraform outputs and AWS CLI rather than relying on the AWS Management Console.

Check	Verification
Terraform outputs	Confirmed instance ID, public IP, DNS, application endpoint, bucket name, VPC, subnet, and IAM profile.
S3 versioning	aws s3api get-bucket-versioning → Status: Enabled.
S3 encryption	aws s3api get-bucket-encryption → AES256.
S3 public access	aws s3api get-public-access-block → all four settings true.
Security group	aws ec2 describe-security-groups → ports 22, 2020
Application	Browser verification of <a href="http://<public_ip>:2020">http://<public_ip>:2020 after Docker Compose startup.
Cleanup	aws s3 ls and aws ec2 describe-instances used after destroy to confirm removal.